

Agentic HPC コード最適化において LLM Reflection は既定の handoff 情報となるべきか 客観的証拠との分離比較

樹神宇徳
名古屋大学大学院情報学研究科

2026年7月31日

要旨

大規模言語モデル (LLM) によるベストファースト木探索 (BFTS) では、子ノードが親のコードに加えて、親 LLM の解釈や次手提案を Reflection として継承する設計が考えられる。しかし、親コードと客観的な評価証拠を既に継承する場合にも、Reflection を常時追加する方針が探索を改善するかは明らかでない。本稿は、この既定方針の妥当性を検証する。

親子間 handoff を **Code only**、**Evidence only**、**Evidence+Reflection** に分け、`gemm`、`spmm`、`stencil` で各条件 30 seed、計 270 の独立 BFTS run を実施した。全 run の独立最終再測定が有効であった。Evidence only – Code only は `stencil` で平均差 +2.90 (95%信頼区間 [1.44, 4.43]、Holm 補正 $p = .00450$) となったが、他の課題では有意でなかった。Evidence+Reflection – Evidence only は 3 課題すべてで有意でなく、効果の符号も揃わなかった。これは Reflection の一般的な無効性や有害性を証明する結果ではないが、客観的証拠へ Reflection を常時追加すれば性能が改善するという仮定を支持しない。Reflection は既定の継承情報ではなく、課題別の検証を要する選択的介入として扱うべきである。

1 はじめに

高性能計算 (High Performance Computing; HPC) に対する LLM コードエージェントは、編集・ビルド・計測を反復しながら候補を探索する [1]。BFTS では高得点ノードを展開し、子は親のコードと workspace を引き継ぐ。ここへ評価結果を追加すれば、親コードの正当性、ケース別性能、失敗理由を明示できる。さらに親 LLM の解釈、懸念、次手提案を Reflection として渡せば、探索を方向付けられる可能性がある。一方、誤った解釈の継承は探索を固定しうる。したがって、言語的指針を増やすことと最適化性能が単調に対応するとは限らない。

特に未検証なのは、客観的証拠が既に与えられた後の Reflection の増分効果である。本研究では、**Reflection** をすべての親子間 handoff に追加する方針は、**Evidence only** より優れた既定値となるかを中心課題とする。併せて、客観的証拠自体の効果と、両者の課題依存性を調べる。比較対象は情報量ではなく、機械的に確認された Evidence と LLM が生成した Reflection という情報の性質である。

本稿の貢献は次の通り。

1. Evidence と Reflection を分離し、Reflection の常時追加だけを変える主要対比を定義する。
2. handoff 方針を run 単位で固定した 270 run の対応実験により、end-to-end の探索性能を評価する。
3. Evidence の効果が課題依存であり、Reflection の常時追加が一貫した利益を示さないことを明らかにする。

2 関連研究

最も近い先行研究 AIDE [2] は ML エンジニアリングをコード空間の木探索とし、各ノードを完全なスクリプトと要約 $\Sigma(T)$ を子へ渡すが、要約中の客観的証拠と LLM 解釈を分離していない。CodeTree [3]、LATS [9]、GIF-MCTS [4]、Tree of Thoughts [8] は critic、scoring、探索

方策を扱うが、handoff の情報成分を比較しない。Reflexion [7] は言語的反省を記憶するのに対し、本研究は客観的証拠が存在する場合の増分効果を測る。すなわち Reflection の有用性全般ではなく、それをすべての handoff に追加する方針が Evidence only を上回るかを問う。KernelBench [5] は GPU カーネルの逐次改良であり、分岐木上の親子継承を対象としない。本研究は、Evidence と Reflection を分離し、run 単位で統制したうえで複数の HPC カーネルを同じ原則で評価する。

3 対象システム

構成 実験システムは、ベストファースト木探索 (BFTS)、各 node 内の ReAct [6] ループ、Model Context Protocol (MCP) で公開するコード操作ツール、固定した非 LLM 評価器からなる。入力は最適化課題と初期 workspace であり、アイデア生成、文献調査、論文生成は実行しない。

run、node、workspace 1 run は、1 課題、1 乱数 seed、1 handoff 条件で実行する独立な BFTS 木を指す。node は木中の 1 回の最適化試行であり、各 node は専用の workspace を持つ。workspace は候補 C ソース、候補用コンパイルフラグ、Makefile、自己テスト用ファイルからなるディレクトリである。各課題の root workspace には、正しい naive 実装と自己テスト用足場を配置する。子 node では、親の候補ソースとビルド設定を複製し、親候補を初期状態として次の最適化を行う。したがって、本稿の“Code”は会話履歴ではなく、この実行可能な workspace を意味する。

1 node の実行 図 1 に処理順を示す。まず実験制御器は、課題記述、利用可能なツールのスキーマ、条件に応じた親 handoff を LLM へ与える。LLM は ReAct 形式で workspace を読み書きし、ビルドと自己テストを反復する。終了後、LLM から独立した非 LLM 評価器が候補だけを別途コンパイルし、正当性と性能を測る。実験制御器は評価値を node のスコアとして保存し、完了 node を展開候補集合 (frontier) に加える。空き探索枠が生じると、BFTS は frontier 中で最高スコアの親を選び、その workspace と、実験条件で許された handoff から子を生成する。

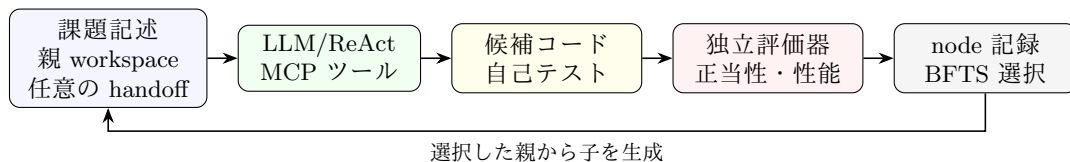


図 1: node 実行と BFTS ループ。評価器はエージェントの自己申告から独立している。

エージェントに公開するツール ツールは MCP サーバから提供し、全課題・全条件で表 1 の 6 個に固定する。各呼出しは node workspace を作業ディレクトリとする。

表 1: LLM エージェントが利用できるツール。

ツール	操作	本実験での位置付け
<code>write_code</code>	指定したテキストをソースファイルへ書く	C ソースや候補用フラグを作成・更新する。コンパイルは行わない。
<code>read_file</code>	テキストファイルを範囲指定して読む	候補、Makefile、自己テスト結果を観察する。
<code>run_code</code>	拡張子に対応するインタプリタでファイルを実行する	Python、shell 等が対象であり、C/C++ 等のコンパイルは行わない。
<code>run_bash</code>	shell コマンドを実行し、標準出力、標準エラー、終了値を返す	コンパイル、リンク、実行、自己テストに用いる。
<code>emit_results</code>	パラメータと測定値を分けた <code>results.json</code> を書く	エージェントの構造化自己記録であり、独立評価器は採点に使用しない。
<code>describe_environment</code>	CPU、GPU、利用可能なコンパイラ、module 等を返す	最適化対象の計算環境を確認する。親の評価値や探索履歴は返さない。

実装ではこの許可リストを条件間で固定し、ツール名、説明、JSON 入力スキーマと提示順も変えない。メモリ検索、評価器呼出し、評価契約生成、ジョブ投入、論文生成の各ツールは LLM に公開しない。特に、採点用ハーネスは MCP ツールではなく、ReAct ループ終了後に実験制御器が呼び出す。

記録と情報経路 各 node の終了時に、実験制御器は `node_report.json` と full trace を保存する。前者は node ID、評価状態、ケース別結果、変更ファイル、LLM の自己記述を生成元別に保持する。さらに `measurement_audit` に各反復の入力 seed、候補・baseline の内部時間と wall time、clamp、残差、採否、実効・拒否コンパイルフラグを保存する。full trace は ReAct の全メッセージ、ツール入出力、および step 上限到達後に Reflection を補完する別 LLM 呼出しの prompt と応答を保存する。これらは監査用であり、許可した Evidence 以外は子へ渡さない。

4 実験設計

探索と handoff 条件 各 node のエージェントは最大 25 ReAct step で上記ツールを用い、候補コードを作る。固定した非 LLM 評価器が全ノードを採点し、BFTS は最高得点のフロンティアノードを展開する（本実験では $N = 10$ ノード/run）。子は常に親 workspace を複製し、追加文脈だけを次の 3 条件で変える（図 2）。

1. **Code only:** workspace のみ。
2. **Evidence only:** workspace と、評価器・ハーネスが確認した客観的証拠。
3. **Evidence+Reflection:** Evidence only に、LLM の解釈・懸念・次手提案を追加。

Evidence+Reflection では、利用可能なすべての親子間 handoff に Reflection を注入する。したがって評価するのは、失敗時だけ用いる選択的 Reflection ではなく、Reflection を既定値とする *always-on* 方針である。1 run には 1 条件だけを適用し、探索木は共有しない。同一 seed の 3 run は共通の乱数列と評価入力を持つ対応ブロックである。ただし親 workspace は共有せず、推定対象は固定親からの局所差ではなく、handoff 方針を通した end-to-end の run 差とする。主要対比は Evidence only – Code only と Evidence+Reflection – Evidence only とする。

workspace の複製では、候補ソース、ビルド設定、自己テスト足場を継承する一方、`results.json`、評価結果、標準出力・標準エラー、生ログ、`node_report.json`、環境プロンプト結果を除く。さらに、過去 node の要約を検索・注入する cross-node memory と、BFTS の子生成プロンプトを作るプランナが親レポートを付与する既定経路を、全条件で無効化する。これにより Code only に別経路から親の測定値や説明が流入することを防ぐ。ただし、親 LLM がソースコメントまたは任意の非除外ファイルへ書いた文章は、コード成果物の一部として残る。

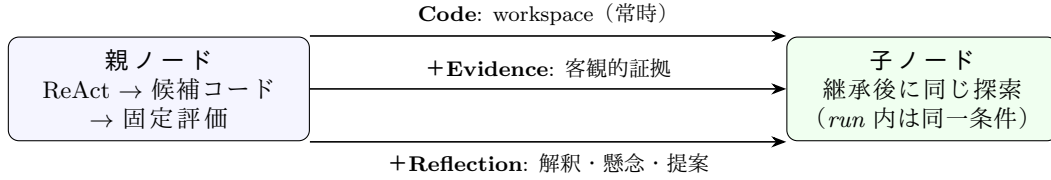


図 2: 親子間 handoff. 各 run は 3 条件のいずれかに固定される。

Evidence と Reflection の生成 表 2 に子プロンプトへ出力するフィールドを示す。Evidence は、制御器、評価器、ハーネス、ファイル hash 比較が機械的に確認した 7 フィールドだけから生成する。evaluation_cases はケースごとの valid と measurements を持ち、測定名は各ハーネスが定義する。known_failures は測定無効時の評価器理由から機械的に導出し、LLM の懸念を参照しない。反復ごとの生時間とコンパイルフラグを持つ measurement_audit は監査専用であり、Evidence へ含めない。

Reflection は、親 LLM が node 終了時に記録した作業解釈、残る懸念、次の最適化案から成る。両 Evidence 条件は同じ見出しとレンダリング関数を使う。同一 report を入力したときの客観部分は byte 単位で同一であり、Evidence+Reflection はその末尾へ Reflection だけを追記する。実際の条件間では探索木が独立なため値自体は異なる。handoff は子の最初の LLM 呼出しに独立メッセージとして注入する。Code only ではこのメッセージを生成せず、raw report、監査記録、full trace は全条件で注入しない。

表 2: Evidence と Reflection の子向けフィールド。上段 7 フィールドは両 Evidence 条件に共通し、下段 3 フィールドは Evidence+Reflection だけに追加する。

子向けフィールド	内容と生成元
Evidence only / Evidence+Reflection 共通	
status	制御器が記録した ReAct 実行状態。候補の正当性判定とは区別する。
measurement_valid	評価器が保存する測定有効フラグ。ビルド・実行・全評価ケースを有効と判定したか。
evaluation_cases	評価ケースごとの valid と、ハーネス定義の客観測定値 measurements。共通スキーマは問題固有の測定名を規定しない。
evaluator_reason	固定評価器が返した採択・棄却理由。
key_metrics	metrics から転記する評価値。幾何平均・ケース別 speedup 等を含む。
changed_files	raw files_changed から転記する、hash 比較済みの追加・変更・削除ファイル。
known_failures	測定無効時の evaluator_reason から機械的に導出。LLM の懸念は含めない。
Evidence+Reflection のみ	
reflection_summary	親 LLM による実施内容とボトルネック解釈の要約。
concerns	親 LLM が自己評価で挙げた未解決の懸念。
next_steps	親 LLM が提案した次の最適化案。

値が存在しないフィールドは出力しない。Code only には表 2 のフィールドを一つも渡さず、出力・ログ・レポートを除外した親 workspace だけを渡す。

課題と採点 3 課題はいずれも、正当性を満たした候補の naive baseline 比 speedup を最大化する (表 3)。

表 3: 主実験の課題。

課題	評価ケースと探索中の反復数	主な最適化特性
gemm	(512, 512, 512), (1024, 256, 256), (256, 256, 1024); 各 3 反復	規則的・計算集約的; データ再利用、SIMD、並列化
spmm	$n = 20000$, $k = 64$; 一様・帯・ベキ則・ブロック・対角優位・偏りの 6 疎構造族; 各 10 反復	不規則・メモリ依存; CSR アクセス、負荷分散
stencil	(256, 256, 256, 30), (384, 192, 192, 30), (192, 192, 384, 30); 各 3 反復	局所性、境界処理、並列化、時間ブロッキング

評価器は候補と凍結 baseline を別々にコンパイルし、fp64 参照解と要素ごとに照合する。出力要素 z の許容誤差は

$$|z_c - z_r| \leq \begin{cases} c_\epsilon \gamma(p) (|A||B|)_{ij}, & \text{gemm,} \\ c_\epsilon \gamma(\text{nnz}_i) (|A||X|)_{ij}, & \text{spmm,} \\ c_\epsilon \gamma(7) \max(n_t, 1) \max |u_0|, & \text{stencil,} \end{cases}$$

とする。ここで $\gamma(k) = ku/(1 - ku)$ 、 u は fp64 の unit roundoff、 c_ϵ は固定安全係数、 p は縮約長、 nnz_i は第 i 行の非零数である。

各反復で新しい入力を生成し、候補と baseline を交互順で対応実行する。奇数反復で一方が1回多くなるため、開始順を seed の奇偶で反転し、30 seed 全体では各条件の実行順を釣り合わせる。候補の内部時間は外部 wall time の下界で検査・clamp し、正当性と時間検査を通ったペアだけを採用する。ケース i の速度比 g_i は採用ペアの t_b/t_c の中央値とし、過半数未満なら無効とする。全評価ケースが有効な場合だけ

$$\text{valid} = \text{compiles} \wedge \bigwedge_i (\text{valid}_i \wedge g_i > 0 \wedge \text{finite}(g_i)),$$

$$g = \begin{cases} \exp\left(k^{-1} \sum_{i=1}^k \log g_i\right), & \text{valid,} \\ 0, & \text{otherwise} \end{cases}$$

と採点する。無効ケースを部分点として幾何平均へ混ぜない。 g は探索中の BFTS 順位付けと候補選択にそのまま用いる。親との hash 差分がない no-op 子は、測定値と valid 判定を客観的記録として保持する一方、探索適格性を示す `_sterile` を付けて再展開、親の retire、最終候補選択から除外する。探索深さ上限は 10 に固定し、root を含む 10 node を各 run に要求する。readable な `node_report.json` が 10 件に満たない run は、条件依存の計算量差として扱わず `infrastructure_error` とする。探索終了後、最大 g の有効な非 no-op 候補を別 seed (run seed + 10^6)、各ケース 15 反復で再測定し、この再測定値を run の主要アウトカムとする。再測定で無効なら 0 とする。

測定防御と最適化フラグ 採点ハーネスと baseline は node workspace 外に置き、manifest の sha256 と一致しない場合は `infrastructure_error` として run を停止する。評価には workspace の候補実装だけを取り込み、各反復を新しい入力・プロセスで実行する。出力は NaN で初期化し、参照解との不一致、短い出力、異常終了、内部時間と wall time の重大な不整合を fail-closed で棄却する。1 プロセスの実行上限は 120 秒に固定し、候補の超過は `candidate_invalid`、凍結 baseline の超過は `infrastructure_error` とする。いずれか1ケースで候補が無効なら、全ケース必須の採点は既に無効なので残りのケースを実行しない。課題契約は外部の既製カーネルへの処理委譲を禁じ、リンク・include 用フラグも拒否する。

候補はコードとコンパイル最適化フラグを探索できるが、baseline は既定フラグに固定する。allowlist は最適化・チューニング用フラグだけを許可し、外部リンク、include path、出力先、ツールチェーンの変更を拒否する。実効・拒否フラグ、評価設定、ハーネスの sha256 を記録する。ただしエージェントと評価器は同一 uid で動作し、OS レベルの隔離はない。この防御は改変検出と結果検証であり、悪意ある候補に対する完全な containment ではない。

モデルと実行環境 モデルは gpt-oss-120b に固定し、OpenAI 互換 API で推論する。1 課題・1 条件・1 seed の run を独立した Slurm array element として sbatch し、各 element の探索、ビルド、計測、採点を fx700 1 ノード (aarch64、48 物理コア、OMP_NUM_THREADS=48) 内で完結させる。計測対象の OpenMP は OMP_PROC_BIND=spread、OMP_PLACES=cores、OMP_DYNAMIC=FALSE に固定する。参照解を計算する制御器側の BLAS は 1 thread に制限し、参照計算の worker が直後の候補計測と競合することを防ぐ。各 run 内の BFTS node は直列実行し、兄弟 node の自己テストと評価を重ねない。array は最大 9 element を並行実行し、条件を特定ノードへ固定しない。低い array index が先に割り当てられる可能性に備え、3 seed ごとに各条件が各 index 位置へ 1 回ずつ現れるよう配置を循環する。候補と baseline は同一 allocation 内で対として測る。job ID、array index、node 名、CPU 数を manifest に保存し、条件別 node 割当も報告する。handoff 関連の外部環境変数は run 起動前に消去し、解決後の設定を記録する。

解析方法と実験凍結 解析単位は 1 課題・1 seed・1 条件の独立 BFTS run である。各課題・各条件を 30 seed で反復し、全体で $3 \times 3 \times 30 = 270$ run とする。主要対比は Evidence only – Code only と Evidence+Reflection – Evidence only の 2 つである。各課題について、同一 seed の run を対応させ、無効な再測定を 0 とした速度比の平均差を 20,000 回の両側 sign-flip permutation test と 5,000 回の seed-pair bootstrap 95%信頼区間で評価する。有効率は exact McNemar 検定と対応リスク差、両条件で有効な seed pair に限定した大きさは log-speedup 差と幾何平均比として分けて報告する。3 課題×2 主要対比の permutation p 値へ Holm 補正を適用する。探索中の親子改善率、親破壊率、node 有効率は副次解析とし、親子率からは no-op 子を除外する。

科学的無効は candidate_invalid または measurement_invalid と記録し、0 を与える。

infrastructure_error は測定値ではない。非零終了と出力欠損も含めて解析から除外し、条件別件数を報告する。主要対比では、いずれか一方が infrastructure error の seed block を除き、除外 seed を明示する。

実験投入前に、実装 commit、3 条件のスキーマ、プロンプト、ツール、モデル、 $N = 10$ 、ReAct 上限、seed 0–29、評価器、ハーネス、解析コードを凍結した。run ごとに実験 driver、解析・batch script、制御コード、課題ハーネス、設定、git status、binary diff と各 hash を bundle 化した。先行 smoke run は実行可能性と収集経路の確認だけに用い、効果推定にも seed 数の決定にも用いていない。投入時に共通 source fingerprint を固定し、各 worker の開始前・終了後と集計時に照合した。不一致 run は解析対象外とした。

5 結果

完了状況と監査 270 run は課題・条件・seed ごとに独立した fx700 allocation で完了し、各 run は要求した 10 node report と 15 反復の独立最終再測定を生成した。全 worker で凍結した source fingerprint が一致し、最終再測定は 270/270 run で有効であった。したがって、科学的無効への 0 代入と infrastructure error による対応 block の除外は、最終解析では発生しなかった。

完了後の API trace 監査で、terminal RateLimitError が BFTS の 1 node 試行を消費した cell を 2 件 (stencil/Code only/seed 7 と spmm/Evidence+Reflection/seed 10) 検出した。観測スコアによらず該当する課題・条件・seed の cell 全体を同じ設定で再実行し、元の 270 cell を変更せず、268 cell と 2 件の再実行を結合した派生データを解析した。再実行には失敗 LLM 呼出しがなく、source fingerprint も一致した。誤った fingerprint で投入された別の事前検査 job は API 呼出し前に停止し、データへ含めていない。

主要アウトカム 表 4 に naive baseline 比 speedup の run 平均を示す。全条件で最終再測定の有効率は 30/30 であり、主要対比の対応リスク差はすべて 0、exact McNemar $p = 1$ であった。

表 4: 独立最終再測定 speedup の算術平均 [bootstrap 95%信頼区間]。各セル $n = 30$ 、全 run 有効。

課題	Code only	Evidence only	Evidence+Reflection
gemm	76.99 [73.38, 80.23]	78.67 [75.77, 81.28]	81.21 [79.56, 82.43]
spmm	57.05 [54.23, 60.41]	54.32 [53.68, 55.06]	56.30 [53.97, 59.31]
stencil	14.29 [12.93, 15.68]	17.18 [16.00, 18.28]	16.11 [14.85, 17.24]

同一 seed を対応させた主要対比を表 5 に示す。Evidence only – Code only は **stencil** で平均差 +2.90、Holm 補正 $p = .00450$ となり、両条件で有効な run における幾何平均比も 1.220 であった。他の 5 対比は Holm 補正後に有意でない。特に Reflection の追加は、点推定が **gemm** で +2.54、**spmm** で +1.98、**stencil** で -1.07 と課題間で符号が揃わず、always-on 方針の一貫した優位性を支持しなかった。両条件で有効な run の幾何平均比 [95% bootstrap 信頼区間] は順に 1.036 [0.999, 1.080]、1.030 [0.989, 1.079]、0.934 [0.826, 1.062] であり、いずれも 1 を含んだ。

表 5: 主要対比。 Δ は対応 seed 内の speedup 平均差、信頼区間は 5,000 回の対応 bootstrap、 p は 20,000 回の両側 sign-flip 検定、Holm p は全 6 検定で補正した値。E-C は Evidence only – Code only、R-E は Evidence+Reflection – Evidence only。

課題	対比	Δ	95%信頼区間	raw p	Holm p
gemm	E-C	+1.68	[-2.68, 6.23]	.482	.642
gemm	R-E	+2.54	[-0.17, 5.51]	.0966	.483
spmm	E-C	-2.72	[-5.90, -0.003]	.115	.483
spmm	R-E	+1.98	[-0.56, 5.06]	.214	.642
stencil	E-C	+2.90	[1.44, 4.43]	.00075	.00450
stencil	R-E	-1.07	[-2.85, 0.80]	.265	.642

spmm の E-C では percentile bootstrap 区間の上端がわずかに 0 を下回る一方、sign-flip $p = .115$ であった。両者は異なる再標本化分布を用いるため、事前に定めた sign-flip 検定と Holm 補正に基づき有意差なしと判断する。RateLimit のあった元 cell を置換せず保持する感度解析でも、Holm 補正後に有意なのは **stencil** の E-C だけであり (Holm $p = .00750$)、結論は変わらなかった。

探索中の親子遷移 表 6 は node 単位の副次解析である。**stencil** では Evidence only が子改善率を 18.9% から 29.7% へ高めた一方、親破壊率も 7.0% から 15.2% へ上昇した。**spmm** でも Evidence 条件の子改善率は Code only より高かったが、最終アウトカムの改善には結び付かなかった。Reflection 追加時の親破壊率は Evidence only より 3 課題すべてで低く、Reflection が親コード破壊を増やすという方向の証拠は得られなかった。ただし node は同一探索木内で依存するため、これらは記述統計である。

表 6: 探索中の node 有効率／子改善率／親破壊率 (%)。有効率の分母は各条件 300 node。子率は、有効親から生じた非 no-op 子 269–270 件を分母とし、改善は親比 > 1.01 、破壊は子の無効化を表す。

課題	Code only	Evidence only	Evidence+Reflection
gemm	98.3/19.6/1.9	96.3/19.6/4.1	97.3/18.5/3.0
spmm	93.3/24.8/7.4	94.3/31.9/6.3	96.3/30.9/3.7
stencil	93.7/18.9/7.0	86.0/29.7/15.2	88.7/26.4/12.3

6 考察

Reflection は既定値となるか Reflection を既定値とするには、同一の客観的証拠を持つ Evidence only に対して、課題を越えて再現する増分効果が必要である。本実験では 3 課題すべてで有意な優位性がなく、点推定も正負に分かれた。したがって、「解釈や次手提案を追加すれ

ば、より豊富な handoff によって探索性能も改善する」という単調な設計仮定は支持されない。これは Reflection 全般の無効性や有害性を意味しない。本実験の対象は、単一モデルが生成した 3 フィールドを毎回追加する方針であり、失敗や探索停滞を契機とする選択的 Reflection と外部検証付き Reflection は別の介入である。Evidence と Reflection は別経路に保ち、後者を無条件に注入せず課題別に検証すべきである。

Evidence の課題依存性 Evidence only は `stencil` で Code only を上回ったが、`gemm` と `spmm` では有意な利益を示さなかった。客観的証拠であっても常に有効とは限らず、子がその情報から有効な次手を導けるかは課題構造に依存する。`spmm` では Evidence 条件で子改善率が高い一方、最終アウトカムは改善しなかった。局所的な親子改善は、固定 node 予算における最良候補の改善を保証しない。

想定した害の検討 Reflection が親コードを破壊するという説明は、本結果に支持されない。Evidence+Reflection の親破壊率は Evidence only より 3 課題すべてで低かった。一方、探索方向の固定化は現在の指標では直接測定していない。本研究が示すのは害の機序ではなく、Reflection の常時追加を支持する最終性能上の根拠が得られなかったという方針レベルの結果である。

7 限界

1. Code only でも、親が workspace に書いたコメントやメモは継承されうる。
2. 同一 uid で動く候補と評価器の間に `syscall`・`filesystem containment` がなく、hash 検査だけでは悪意ある改変を完全には防げない。
3. RateLimit cell の全体置換規則は完了後監査で明文化した。スコア非依存の一貫した規則を適用し、元 cell を保持した感度解析も併記したが、事前指定ではない。
4. 非有意差は効果ゼロの証明ではない。同等性幅を事前指定しておらず、結果も単一モデル、Reflection 形式、3 カーネル、計算環境に依存するため、Reflection の一般的な無効性は主張しない。
5. 親子改善・破壊率は探索木内で依存し、探索方向の固定化を直接測る指標ではない。
6. 性能測定には残留ノイズがあり、選択的または外部検証付き Reflection も評価していない。

8 結論

親コード継承下の handoff を Code only、Evidence only、Evidence+Reflection に分解し、3 HPC カーネル、270 run で比較した。Evidence の効果は `stencil` に限って確認され、客観的情報の価値も課題依存であった。さらに Reflection を常時追加する方針は Evidence only を 3 課題で一貫して上回らなかった。これは Reflection 全般の否定ではなく、本設定ではそれを既定の継承情報とする根拠が得られなかったことを示す。Evidence と Reflection を分離し、後者を課題別に検証する介入として扱うことが、親子間 handoff の妥当な設計原則である。

参考文献

- [1] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, D. Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv:2408.06292*, 2024.
- [2] Z. Jiang, D. Schmidt, D. Srikanth, et al. (Weco AI). AIDE: AI-Driven Exploration in the Space of Code. *arXiv:2502.13138*, 2025.
- [3] J. Li, G. Li, X. Zhang, et al. CodeTree: Agent-guided tree search for code generation with large language models. *NAACL* (arXiv:2411.04329), 2025.
- [4] N. Dainese, M. Merler, M. Alakuijala, P. Marttinen. Generating code world models with reinforcement learning and Monte Carlo Tree Search (GIF-MCTS). *NeurIPS* (arXiv:2405.15383), 2024.
- [5] A. Ouyang, S. Guo, S. Arora, et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv:2502.10517*, 2025.
- [6] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [7] N. Shinn, F. Cassano, A. Gopinath, et al. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.
- [8] S. Yao, D. Yu, J. Zhao, et al. Tree of Thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023.
- [9] A. Zhou, K. Yan, M. Shlapentokh-Rothman, et al. Language Agent Tree Search unifies reasoning, acting, and planning in language models. *ICML* (arXiv:2310.04406), 2024.